# Plan de estudio para la defensa

## Logística de última milla

El código del sistema explicado en cinco partes, una por integrante

Trabajo Práctico Integrador 2026 · Backend de Aplicaciones · UTN Facultad Regional Córdoba

| Parte | Presenta | Tema |
|---|---|---|
| 1 | **Walter** | Arquitectura, API Gateway y seguridad |
| 2 | **Fran** | pedidos-service: clientes, pedidos y ciclo de vida |
| 3 | **Tobi** | despacho-service: planificación de rutas y distancias |
| 4 | **Nacho** | Ejecución de la ruta y colaboración entre servicios |
| 5 | **Tomi** | Seguimiento, mails, persistencia, despliegue y pruebas |

Noviembre de 2026

# Índice


**Cómo usar este documento** **4**

Pasos de estudio 4

Cómo está armada cada parte 4

**Lo que todo el equipo tiene que saber** **5**

El problema, en un minuto 5

El recorrido de un pedido 5

Cinco ideas que sostienen el diseño 5

Qué mira la rúbrica y quién lo defiende 5

**Parte 1 · Walter: Arquitectura, API Gateway y seguridad** **7**

Archivos que la componen 7

Mapa de la arquitectura 7

Fragmentos clave 8

Matriz de permisos 9

Conceptos clave 10

Preguntas probables 10

**Parte 2 · Fran: pedidos-service: clientes, pedidos y ciclo de vida** **11**

Archivos que la componen 11

Ciclo de vida del pedido 11

Fragmentos clave 12

Códigos de respuesta de pedidos-service 15

Conceptos clave 15

Preguntas probables 16

**Parte 3 · Tobi: despacho-service: planificación de rutas y distancias** **17**

Archivos que la componen 17

El proceso, paso a paso 17

Fragmentos clave 18

Conceptos clave 20

Preguntas probables 21

**Parte 4 · Nacho: Ejecución de la ruta y colaboración entre servicios** **22**

Archivos que la componen 22

Quién llama a quién, y por qué 22

Fragmentos clave 23

Estados de la ruta y de la parada 26

Conceptos clave 26

Preguntas probables 26

**Parte 5 · Tomi: Seguimiento, mails, persistencia, despliegue y pruebas** **28**

Archivos que la componen 28
Fragmentos clave 29
Persistencia 32
Pruebas 32
Conceptos clave 33
Preguntas probables 33
**Antes de la defensa** **34**
Lista de verificación 34
Límites conocidos 34
Cómo responder una pregunta que no es de tu parte 34

# Cómo usar este documento

| Parte | Presenta | Tema | Rúbrica que lidera | Demo (Postman) |
|---|---|---|---|---|
| 1 | **Walter** | Arquitectura, API Gateway y seguridad | ARQ-01, SEG-01 | 1. Autenticación y datos maestros |
| 2 | **Fran** | pedidos-service: clientes, pedidos y ciclo de vida | NEG-02, TEC-01 | 2. Cliente: perfil y pedidos |
| 3 | **Tobi** | despacho-service: planificación de rutas y distancias | NEG-01, DES-01 | 3. Planificación (despachante) |
| 4 | **Nacho** | Ejecución de la ruta y colaboración entre servicios | INT-01 | 4. Ejecución de la ruta (repartidor) |
| 5 | **Tomi** | Seguimiento, mails, persistencia, despliegue y pruebas | INF-01, API-01, ALC-01 | 5. Seguimiento, mails y seguridad |

## Pasos de estudio

1. **Todos** leen la sección «Lo que todo el equipo tiene que saber». La rúbrica (PRE-01) pide que cualquier integrante pueda explicar el sistema, no solo su parte.
2. Cada uno estudia su parte con el código abierto: abrir cada archivo de la tabla, ubicar el fragmento y leer lo que lo rodea.
3. Correr el apartado de Postman que le toca y poder decir, en cada request, qué clase lo atiende y qué regla se está viendo.
4. Responder en voz alta las «Preguntas probables» sin leer. Después, leer las preguntas de la parte anterior y de la siguiente: las partes están encadenadas.
5. Ensayo general: la colección completa en orden, con la base limpia, cada uno presentando su apartado.

## Cómo está armada cada parte

- **Ficha:** quién presenta, objetivo, criterios de la rúbrica que cubre y apartado de Postman para la demo.
- **Archivos que la componen:** qué abrir en el IDE y para qué sirve cada archivo.
- **Fragmentos clave:** el código más importante de cada archivo, con la explicación debajo. Los `...` indican líneas omitidas.
- **Conceptos clave:** la teoría que hay que poder explicar con palabras propias.
- **Preguntas probables:** lo que puede preguntar el docente, con una respuesta corta.

El orden de las partes sigue el recorrido de un pedido: entra por el gateway, se carga, se planifica, se entrega y se le hace seguimiento. Presentar en ese orden cuenta el proceso de negocio de punta a punta.

# Lo que todo el equipo tiene que saber

## El problema, en un minuto

Una empresa de logística recibe pedidos de comercios y tiene que entregarlos al destinatario final. Cada día, un despachante debe decidir qué pedidos van en qué vehículo, con qué repartidor y en qué orden, respetando capacidad, cadena de frío, horarios de entrega, turnos y autonomía, al menor costo. Después hay que seguir lo que pasa en la calle: entregas, intentos fallidos, reintentos y devoluciones, y avisarle al comercio en cada paso.

## El recorrido de un pedido

| # | Qué pasa | Quién | Servicio | Estado del pedido | Parte |
|---|---|---|---|---|---|
| 1 | Inicia sesión y obtiene un token | Cualquier usuario | Keycloak | — | 1 |
| 2 | Registra su perfil y carga un pedido | CLIENTE | pedidos (consulta el centro a despacho) | RECIBIDO | 2 |
| 3 | Planifica el día: se arman las rutas | DESPACHANTE | despacho (pide pendientes y confirma en pedidos) | ASIGNADO o NO_ASIGNABLE | 3 |
| 4 | Inicia la ruta | REPARTIDOR | despacho (avisa a pedidos por outbox) | EN_CAMINO | 4 |
| 5 | Registra cada parada | REPARTIDOR | despacho (avisa a pedidos por outbox) | ENTREGADO, o FALLIDO → RECIBIDO / DEVUELTO | 4 |
| 6 | Consulta el estado y el historial; recibe un mail por cambio | CLIENTE | pedidos | — | 5 |

## Cinco ideas que sostienen el diseño

- **Dos servicios, dos responsabilidades:** pedidos-service es dueño del pedido y su estado; despacho-service es dueño de la flota y de las rutas. Cada uno con su base.
- **Un solo punto de entrada:** todo pasa por el gateway (8080) con un token de Keycloak. Lo interno entre servicios no se puede alcanzar desde afuera.
- **Las reglas están en un lugar reconocible:** las del pedido en la entidad `Pedido`; las de la ruta en `CalculadoraRuta` y `PlanificadorRutas`.
- **Se pensó qué pasa cuando algo falla:** despacho caído → no se crea el pedido (503); pedidos caído → las rutas quedan en borrador y los avisos en el outbox; OSRM caído → caché o estimación; Gmail caído → el pedido sigue.
- **Todo es comprobable:** más de 90 tests automáticos y una colección de Postman con verificaciones que recorre el proceso completo.

## Qué mira la rúbrica y quién lo defiende

| Criterio | Pts | Qué hay que mostrar | Lidera | Evidencia |
|---|---|---|---|---|
| ARQ-01 Arquitectura | 12 | Servicios con responsabilidades del dominio; recorrido de una operación | Walter | Diagramas C1 y C2 del README; mapa de la parte 1 |
| SEG-01 Seguridad | 8 | Matriz de permisos; casos permitidos y denegados | Walter | Matriz de la parte 1; requests 401 y 403 de la colección |
| TEC-01 Calidad técnica | 10 | Capas, validación, errores controlados, pruebas significativas | Fran (pruebas: Tomi) | Código de pedidos-service; `mvn test` |

| Criterio | Pts | Qué hay que mostrar | Lidera | Evidencia |
|---|---|---|---|---|
| INF-01 Persistencia y despliegue | 10 | Modelo de datos, configuración, Docker, arranque desde cero | Tomi | `docker compose up`; README |
| NEG-01 Proceso central | 15 | Flujo completo con estados y resultado verificable | Tobi (todos) | Colección completa en orden |
| NEG-02 Reglas de negocio | 10 | Reglas trazables a casos de prueba; casos normales y rechazos | Fran y Tobi | Tabla RN01–RN20 del README |
| INT-01 Integración | 12 | Quién llama a quién, contratos, error de integración controlado | Nacho | Tabla de la parte 4; demo con un servicio detenido |
| ALC-01 Alcance | 8 | Matriz compromiso → implementación → evidencia; desvíos | Tomi | Secciones «Trazabilidad» y «Desvíos» del README |
| DES-01 Desafío | 5 | Por qué el problema es complejo y qué se resolvió | Tobi | Heurística, restricciones simultáneas, saga |
| PRE-01 Presentación | 5 | Exposición ordenada; todos defienden con solvencia | Todos | Ensayo general |
| API-01 Colección | 5 | Colección organizada, parametrizada, con negativos y verificaciones | Tomi | `postman/` |
| DOC-01 Apreciación global | 5 | Proceso de trabajo, evolución, responsabilidad compartida | Todos | Historial de commits; desvíos informados |

# Parte 1 · Walter: Arquitectura, API Gateway y seguridad

| Presenta | **Walter** |
|---|---|
| Objetivo | Explicar cómo está partido el sistema, por dónde entra cada request y cómo se decide quién puede hacer qué. |
| Criterios de la rúbrica | ARQ-01 (12 pts) y SEG-01 (8 pts). |
| Demo en Postman | Apartado **1. Autenticación y datos maestros**: login de cada rol, consulta de la semilla, alta y baja de un vehículo. |

## Archivos que la componen

| Archivo | Qué hace |
|---|---|
| `api-gateway/src/main/resources/application.yml` | Rutas del gateway (prefijo → servicio) y dónde validar el JWT. |
| `api-gateway/.../gateway/SecurityConfig.java` | Filtro de seguridad del gateway: exige token en todo y bloquea `/internal/**`. |
| `infra/realm-logistica.json` | Realm de Keycloak: 5 roles, 2 clients y 6 usuarios de prueba. |
| `logistica-commons/.../KeycloakRoleConverter.java` | Convierte los roles del token en autoridades de Spring Security. Lo usan los dos servicios. |
| `pedidos-service/.../config/SecurityConfig.java` | Resource server de pedidos; los roles finos van con `@PreAuthorize` en los controllers. |
| `despacho-service/.../config/SecurityConfig.java` | Resource server de despacho; reglas por URL y método HTTP. |
| `despacho-service/.../security/Roles.java` | Constantes con los nombres de los roles. |
| `despacho-service/.../controller/{Centro,Vehiculo,Repartidor}Controller.java` | Datos maestros (ABM) que se muestran en la demo de esta parte. |
| `PedidoControllerSecurityTest.java`, `DespachoSecurityTest.java` | Tests de seguridad: sin token, rol incorrecto, recurso ajeno. |

## Mapa de la arquitectura

| Componente | Puerto | Responsabilidad | Datos propios |
|---|---|---|---|
| api-gateway | 8080 | Único punto de entrada: enruta, valida el JWT, bloquea lo interno. | Ninguno |
| pedidos-service | 8081 | Clientes y pedidos: alta, ciclo de vida, cancelación, historial, mails. | `pedidos_db` |
| despacho-service | 8082 | Operación: centros, flota, planificación de rutas, ejecución, distancias. | `despacho_db` |
| logistica-commons | JAR | Lo que es igual en los dos servicios: errores, roles, `Coordenada`. | — |
| Keycloak | 8180 | Autentica usuarios, emite los JWT, administra roles. | `keycloak_db` |

**Por qué dos servicios:** el pedido lo carga el cliente y su historia le interesa al cliente; la operación (flota, rutas, calle) la maneja la empresa. Cada uno guarda solo lo suyo: despacho no tiene los datos personales del destinatario, solo coordenadas, peso y ventana. Ninguno lee la base del otro; se hablan por HTTP en `/internal/**`.

## Fragmentos clave

api-gateway/src/main/resources/application.yml

```
spring:
  cloud:
    gateway:
      server:
        webflux:
          routes:
            - id: pedidos-service
              uri: ${PEDIDOS_SERVICE_URL:http://localhost:8081}
              predicates:
                - Path=/api/pedidos/**
            - id: pedidos-service-clientes
              uri: ${PEDIDOS_SERVICE_URL:http://localhost:8081}
              predicates:
                - Path=/api/clientes/**
            - id: despacho-service
              uri: ${DESPACHO_SERVICE_URL:http://localhost:8082}
              predicates:
                - Path=/api/despacho/**
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: ${KEYCLOAK_ISSUER_URI:http://localhost:8180/realms/logistica}
          jwk-set-uri: ${KEYCLOAK_JWK_SET_URI:http://localhost:8180/realms/logistica/protocol/openid-connect/certs}
```

> El cliente solo conoce el puerto 8080. No hay ninguna ruta hacia `/internal`. `issuer-uri` es el emisor que debe traer el token; `jwk-set-uri` es de dónde se bajan las claves públicas para verificar la firma. Cada valor se puede pisar con una variable de entorno (así funciona igual en local y en Docker).

api-gateway/.../gateway/SecurityConfig.java

```
http
    .csrf(ServerHttpSecurity.CsrfSpec::disable)
    .authorizeExchange(exchanges -> exchanges
            .pathMatchers(HttpMethod.GET, "/actuator/health/**").permitAll()
            // Los endpoints internos entre servicios nunca se exponen al exterior.
            .pathMatchers("/internal/**", "/api/*/internal/**").denyAll()
            .anyExchange().authenticated())
    .oauth2ResourceServer(oauth2 -> oauth2.jwt(Customizer.withDefaults()));
```

> El gateway **autentica** (¿el token es válido?) pero no decide permisos por rol: eso lo hace cada servicio. Sin token responde 401.

logistica-commons/.../KeycloakRoleConverter.java

```
public Collection<GrantedAuthority> convert(Jwt jwt) {
    Map<String, Object> realmAccess = jwt.getClaim("realm_access");
    if (realmAccess == null || !(realmAccess.get("roles") instanceof Collection<?> roles)) {
        return List.of();
    }
    return roles.stream()
            .map(String::valueOf)
            .map(rol -> (GrantedAuthority) new SimpleGrantedAuthority("ROLE_" + rol.toUpperCase()))
            .toList();
}
```

Keycloak guarda los roles en el claim `realm_access.roles`; Spring espera autoridades `ROLE_X`. Sin este conversor, `hasRole('CLIENTE')` nunca daría verdadero. Está en el JAR común para no repetirlo en cada servicio.

despacho-service/.../config/SecurityConfig.java

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/v3/api-docs/**", "/swagger-ui/**", "/actuator/health/**").permitAll()
    // API interna: sólo cuentas de servicio (client credentials)
    .requestMatchers("/internal/**").hasRole(Roles.SERVICE)
    // Datos maestros: lectura para despachante y admin, escritura sólo admin
    .requestMatchers(HttpMethod.GET, API + "/centros/**", API + "/vehiculos/**",
            API + "/repartidores/**").hasAnyRole(Roles.ADMIN, Roles.DESPACHANTE)
    .requestMatchers(API + "/centros/**", API + "/vehiculos/**",
            API + "/repartidores/**").hasRole(Roles.ADMIN)
    .requestMatchers(API + "/mi-ruta/**").hasRole(Roles.REPARTIDOR)
    .requestMatchers(API + "/planificaciones/**").hasRole(Roles.DESPACHANTE)
    .requestMatchers(HttpMethod.GET, API + "/rutas/**").hasAnyRole(Roles.DESPACHANTE, Roles.ADMIN)
    .anyRequest().denyAll())
```

Las reglas se evalúan en orden: la primera que coincide gana. Lo que no está listado se niega (`denyAll`). En pedidos-service la misma idea está con `@PreAuthorize` sobre cada método del controller.

## Matriz de permisos

| Recurso | CLIENTE | DESPACHANTE | REPARTIDOR | ADMIN | SERVICE |
|---|---|---|---|---|---|
| Perfil propio (`/api/clientes/me`) | ✓ | | | | |
| Crear pedido | ✓ | | | | |
| Ver, cancelar pedido e historial | solo los suyos | ✓ | | ✓ | |
| Reprogramar pedido, listar clientes | | ✓ | | ✓ | |
| Datos maestros: leer | | ✓ | | ✓ | |
| Datos maestros: crear, modificar, borrar | | | | ✓ | |
| Planificar | | ✓ | | | |
| Consultar rutas, distancias, avisos | | ✓ | | ✓ | |
| Mi ruta: ver, iniciar, registrar parada | | | solo la suya | | |
| `/internal/**` | | | | | ✓ |

## Conceptos clave

- **OAuth2 / OIDC:** el usuario se autentica contra Keycloak, no contra nuestros servicios. Keycloak devuelve un **JWT** firmado.
- **JWT:** tres partes (cabecera, datos, firma). Claims que usamos: `iss` (emisor), `sub` (id del usuario), `exp` (vencimiento), `realm_access.roles`, `azp` (client que lo pidió).
- **Resource server:** el servicio no guarda sesiones ni contraseñas; valida la firma del token con la clave pública de Keycloak en cada request (*stateless*). Por eso CSRF está desactivado: no hay cookies de sesión.
- **401 vs 403:** 401 = no sé quién sos (sin token o token inválido); 403 = sé quién sos pero no tenés permiso.
- **Defensa en profundidad:** el gateway valida el token, cada servicio lo vuelve a validar y aplica roles, y la capa de servicio controla las reglas que dependen de datos (el cliente solo lo suyo, el repartidor solo su ruta).
- **Dos formas de pedir token:** *password* (client público `logistica-public`, lo usa Postman para los usuarios) y *client credentials* (client `logistica-service`, lo usan los servicios entre sí, con rol `SERVICE`).
- **Identidad desde el token:** el id del usuario sale siempre del claim `sub`, nunca de un parámetro de la URL.

## Preguntas probables

- **¿Por qué el gateway no revisa los roles?** Para que la regla esté en un solo lugar: junto al endpoint que protege. El gateway no conoce el dominio. Además, si alguien llega a un servicio sin pasar por el gateway, el servicio igual lo frena.
- **¿Qué pasa si llamo a `/internal/pedidos/pendientes` por el gateway?** El gateway no tiene ruta y además lo niega explícitamente (`denyAll`). Y aunque alguien llegara directo al servicio, necesitaría un token con rol `SERVICE`, que ningún usuario humano tiene.
- **¿Dónde están los secretos?** Cada secreto es una propiedad `${VARIABLE:valor}`. El valor por defecto es de prueba (condición de la cátedra: que corra sin configurar nada). En un entorno real se pasa la variable; en Docker, por el archivo `.env`, que no se versiona.
- **¿Cómo sabe el servicio que el token no fue alterado?** Verifica la firma con la clave pública que baja de `jwk-set-uri`, y además controla el emisor (`iss`) y el vencimiento (`exp`).
- **¿Por qué una librería común y no un tercer microservicio?** Porque no es una capacidad de negocio ni tiene datos: es código repetido (formato de error, conversión de roles, coordenadas). Como JAR se versiona y cada servicio lo incluye al compilar.

# Parte 2 · Fran: pedidos-service: clientes, pedidos y ciclo de vida

| Presenta | **Fran** |
|---|---|
| Objetivo | Explicar cómo se carga un pedido, qué se valida, qué estados puede tener y cómo se devuelven los errores. |
| Criterios de la rúbrica | NEG-02 (10 pts) y TEC-01 (10 pts); aporta a NEG-01. |
| Demo en Postman | Apartado **2. Cliente: perfil y pedidos**: perfil, alta de cuatro pedidos, listado y un pedido inválido rechazado con 422. |

## Archivos que la componen

| Archivo (pedidos-service) | Qué hace |
|---|---|
| `model/Pedido.java` | Entidad central. Tiene las **reglas de negocio**: transiciones, cancelación, reintentos. |
| `model/EstadoPedido.java, PrioridadPedido.java` | Enumeraciones de estado y prioridad. |
| `model/CambioEstado.java` | Registro inmutable de cada cambio de estado (historial). |
| `model/Cliente.java, Destinatario.java` | Perfil del comercio; destinatario embebido en el pedido. |
| `dto/*.java` | Records de entrada y salida. La validación de formato está acá (Bean Validation). |
| `controller/PedidoController.java, ClienteController.java` | API REST pública: rutas, roles, códigos HTTP y documentación OpenAPI. |
| `service/PedidoService.java, ClienteService.java` | Casos de uso: permisos por dato, persistencia e integración. |
| `service/*Exception.java` | Excepciones propias (no encontrado, acceso denegado, conflicto, servicio caído). |
| `config/PedidosExceptionHandler.java` | Traduce las excepciones propias a códigos HTTP. |
| `logistica-commons/.../GlobalExceptionHandler.java, ApiError.java` | Manejo de errores común y formato único de error. |
| `PedidoTest (17), PedidoServiceTest (15), ClienteServiceTest (5)` | Tests de reglas y de casos de uso. |

## Ciclo de vida del pedido

| Desde | Puede pasar a | Quién lo dispara |
|---|---|---|
| RECIBIDO | ASIGNADO, NO_ASIGNABLE, CANCELADO | Planificación (despacho) o cancelación |
| ASIGNADO | EN_CAMINO, CANCELADO | El repartidor inicia la ruta, o cancelación |
| NO_ASIGNABLE | RECIBIDO, CANCELADO | El despachante reprograma o cancela |
| EN_CAMINO | ENTREGADO, FALLIDO | El repartidor registra el resultado |
| FALLIDO | RECIBIDO (1er intento), DEVUELTO (2do) | Automático, dentro de `registrarFallo` |
| ENTREGADO | DEVUELTO | Caso particular |
| DEVUELTO, CANCELADO | — | Estados finales |

## Fragmentos clave

pedidos-service/.../model/Pedido.java

```
private static final int MAX_INTENTOS = 2;

private static final Map<EstadoPedido, Set<EstadoPedido>> TRANSICIONES = Map.of(
        EstadoPedido.RECIBIDO, Set.of(EstadoPedido.ASIGNADO, EstadoPedido.NO_ASIGNABLE, EstadoPedido.CANCELADO),
        EstadoPedido.ASIGNADO, Set.of(EstadoPedido.EN_CAMINO, EstadoPedido.CANCELADO),
        EstadoPedido.NO_ASIGNABLE, Set.of(EstadoPedido.RECIBIDO, EstadoPedido.CANCELADO),
        EstadoPedido.EN_CAMINO, Set.of(EstadoPedido.ENTREGADO, EstadoPedido.FALLIDO),
        EstadoPedido.FALLIDO, Set.of(EstadoPedido.RECIBIDO, EstadoPedido.DEVUELTO),
        EstadoPedido.ENTREGADO, Set.of(EstadoPedido.DEVUELTO),
        EstadoPedido.DEVUELTO, Set.of(),
        EstadoPedido.CANCELADO, Set.of());

/** Unico punto de modificacion del estado. */
public void cambiarEstado(EstadoPedido nuevo, String motivo, String usuario) {
    // ... valida que nuevo y usuario no sean nulos
    if (!TRANSICIONES.getOrDefault(estado, Set.of()).contains(nuevo)) {
        throw new IllegalArgumentException("Transicion no permitida: " + estado + " -> " + nuevo);
    }
    EstadoPedido anterior = this.estado;
    this.estado = nuevo;
    this.cambios.add(new CambioEstado(this, anterior, nuevo, motivo, usuario));
}
```

La máquina de estados es una tabla, no una cadena de `if`. El campo `estado` no tiene setter público (`@Setter(AccessLevel.NONE)`): la única forma de cambiarlo es este método, que además deja el registro en el historial. Así es imposible cambiar de estado sin dejar rastro (RN05).

pedidos-service/.../model/Pedido.java

```
public void registrarFallo(String motivo, String usuario) {
    if (estado != EstadoPedido.EN_CAMINO) {
        throw new IllegalArgumentException(
                "Solo se puede registrar un fallo desde EN_CAMINO, estado actual: " + estado);
    }
    intentos++;
    cambiarEstado(EstadoPedido.FALLIDO, motivo, usuario);
    if (intentos >= MAX_INTENTOS) {
        cambiarEstado(EstadoPedido.DEVUELTO, "Maximo de intentos alcanzado (" + intentos + ")", USUARIO_SISTEMA);
    } else {
        cambiarEstado(EstadoPedido.RECIBIDO, "Reintento automatico (intento " + intentos + ")", USUARIO_SISTEMA);
    }
}
```

RN16: el primer intento fallido devuelve el pedido a la cola (RECIBIDO) para la próxima planificación; el segundo lo da por DEVUELTO. La decisión la toma la entidad, no el controller ni despacho.

pedidos-service/.../service/PedidoService.java

```
@Transactional
public PedidoDTO crear(String keycloakUserId, CrearPedidoRequest request) {
    // 1. perfil
    Cliente cliente = clienteRepository.findByKeycloakUserId(keycloakUserId)
            .orElseThrow(() -> new IllegalArgumentException("Debe registrar su perfil de cliente"));
    // 2. cliente habilitado (regla del modelo)
    if (!cliente.puedeCrearPedidos()) {
        throw new IllegalArgumentException("El cliente esta desactivado y no puede crear pedidos");
    }
    Pedido pedido = new Pedido();
    // ... copia los datos del request
    // 3. ventana (regla del modelo)
    if (!pedido.ventanaValida()) {
        throw new IllegalArgumentException("La ventana de entrega es invalida: ...");
    }
    // 4. centro: 422 si no existe o esta inactivo, 503 si despacho no responde
    CentroDTO centro = despachoClient.obtenerCentro(request.centroId());
    if (!centro.activo()) {
        throw new IllegalArgumentException("El centro de distribucion " + centro.id() + " esta inactivo");
    }
    return PedidoDTO.from(pedidoRepository.save(pedido));
}
```

El servicio orquesta; las reglas viven en el modelo (`puedeCrearPedidos`, `ventanaValida`). El paso 4 es una llamada real a despacho-service: el centro es un dato de despacho, así que se le pregunta a su dueño.

pedidos-service/.../controller/PedidoController.java

```
@PostMapping
@PreAuthorize("hasRole('CLIENTE')")
@Operation(summary = "Crea un pedido para el cliente autenticado")
@ApiResponses({
        @ApiResponse(responseCode = "201", description = "Pedido creado (header Location)"),
        @ApiResponse(responseCode = "400", description = "Datos invalidos"),
        @ApiResponse(responseCode = "422", description = "Sin perfil, cliente desactivado, ventana invalida o centro
inexistente/inactivo"),
        @ApiResponse(responseCode = "503", description = "despacho-service no responde: el pedido no se crea")
})
public ResponseEntity<PedidoDTO> crear(@AuthenticationPrincipal Jwt jwt,
                                       @Valid @RequestBody CrearPedidoRequest request,
                                       UriComponentsBuilder uriBuilder) {
    PedidoDTO creado = pedidoService.crear(jwt.getSubject(), request);
    return ResponseEntity
            .created(uriBuilder.path("/api/pedidos/{id}").buildAndExpand(creado.id()).toUri())
            .body(creado);
}

/** true cuando el usuario solo puede ver y operar sus propios pedidos. */
private boolean soloPropios(Authentication authentication) {
    return authentication.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .noneMatch(ROLES_GLOBALES::contains);   // ROLE_DESPACHANTE, ROLE_ADMIN
}
```

El controller es fino: rol, validación (`@Valid`), identidad desde el token (`jwt.getSubject()`) y código HTTP. Devuelve 201 con cabecera `Location`. Las anotaciones `@Operation` y `@ApiResponse` generan la documentación de Swagger.

pedidos-service/.../dto/CrearPedidoRequest.java

```
public record CrearPedidoRequest(
        @NotNull Long centroId,
        @NotNull @Valid DestinatarioDTO destinatario,
        @NotNull @Positive Double pesoKg,
        @NotNull @Positive Double volumenM3,
        @NotNull PrioridadPedido prioridad,
        @NotNull LocalDateTime ventanaDesde,
        @NotNull LocalDateTime ventanaHasta,
        boolean requiereFrio
) {
}
```

Validación de formato con Bean Validation: si falla, ni siquiera se entra al servicio y la respuesta es 400. Las reglas que dependen de datos (ventana invertida, centro inexistente) dan 422.

logistica-commons/.../GlobalExceptionHandler.java

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    /** Fallo de validacion de @Valid -> 400. */
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiError> handleValidacion(...) { ... }

    /** Argumento o regla de negocio invalida -> 422. */
    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<ApiError> handleNegocio(IllegalArgumentException ex, WebRequest request) {
        return build(HttpStatus.UNPROCESSABLE_ENTITY, ex.getMessage(), request);
    }

    /** Cualquier otro error -> 500. */
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiError> handleGenerico(...) { ... }
}
```

> Ningún controller tiene `try/catch`: las excepciones suben y un único componente las convierte en respuestas con el mismo formato (`ApiError`: timestamp, status, error, mensaje, path).

## Códigos de respuesta de pedidos-service

| Código | Cuándo | Ejemplo |
|---|---|---|
| 201 / 200 | Operación correcta | Alta de pedido, consulta |
| 400 | Datos con formato inválido | Peso negativo, campo faltante |
| 401 | Sin token o token inválido | Request sin cabecera `Authorization` |
| 403 | Rol insuficiente o recurso ajeno | Un cliente pide el pedido de otro |
| 404 | El recurso no existe | Pedido inexistente, perfil sin crear |
| 409 | Conflicto con datos existentes | Perfil o CUIT ya registrado |
| 422 | Regla de negocio no cumplida | Ventana invertida, transición no permitida |
| 503 | despacho-service no responde | Alta de pedido con despacho caído |

## Conceptos clave

- **Capas:** controller (HTTP) → service (caso de uso, transacción) → model (reglas) → repository (datos). Los DTO separan lo que se expone de lo que se guarda.
- **Modelo rico:** las reglas están en la entidad (`cancelar`, `reprogramar`, `registrarFallo`), no dispersas en los servicios. Se prueban sin Spring ni base de datos (`PedidoTest`).
- **Bean Validation** (`@NotNull`, `@Positive`, `@Email`) + `@Valid` para el formato; excepciones de negocio para las reglas.
- **`@Transactional`:** el caso de uso se confirma entero o no se confirma. Al modificar una entidad cargada, Hibernate guarda el cambio solo al terminar (no hace falta llamar a `save`).
- **Propiedad del dato:** el CLIENTE solo ve lo suyo (`Pedido.perteneceA` compara con el `sub` del token). Pedido ajeno → 403.
- **Referencia lógica:** `centroId` es solo un número; no hay clave foránea hacia la base de despacho.
- **Listado:** paginado (`Pageable`) y con filtros opcionales armados con `Specification`.

## Preguntas probables

- **¿Por qué 422 y no 400 para una ventana invertida?** 400 es para un request mal formado; acá el JSON es correcto pero viola una regla del dominio. Separarlos le permite al cliente distinguir «arreglá el formato» de «esto no se puede hacer».
- **¿Cómo garantizan que no haya una transición inválida?** El estado solo se cambia por `cambiarEstado`, que consulta la tabla `TRANSICIONES`. No hay setter. `PedidoTest` prueba transiciones válidas e inválidas.
- **¿Dónde está la lógica de negocio: en el service o en la entidad?** En la entidad lo que es regla del pedido; en el service lo que es coordinación (buscar, verificar acceso, llamar a despacho, guardar).
- **¿Qué pasa si despacho-service está caído cuando se crea un pedido?** El pedido no se crea y se responde 503. Se prefirió eso a aceptar un pedido para un centro que no se pudo verificar.
- **¿Por qué el destinatario no es una entidad aparte?** No tiene vida propia fuera del pedido: es un `@Embeddable` y sus columnas están en la tabla `pedido`.
- **Los errores de pedidos y de despacho no tienen el mismo formato. ¿Por qué?** Es un límite conocido: pedidos usa `ApiError` del JAR común y regla de negocio = 422; despacho usa `ProblemDetail` (RFC 7807) con un campo `codigo` y regla de negocio = 409. Cada servicio es consistente por dentro; unificarlos es una mejora pendiente.

# Parte 3 · Tobi: despacho-service: planificación de rutas y distancias

| Presenta | **Tobi** |
|---|---|
| Objetivo | Explicar el proceso central: cómo, a partir de los pedidos pendientes, se arman rutas que respetan todas las restricciones al menor costo. |
| Criterios de la rúbrica | NEG-01 (15 pts), DES-01 (5 pts) y NEG-02 (10 pts). |
| Demo en Postman | Apartado **3. Planificación (despachante)**: planificar, pedido no asignable y reprogramación, cancelación de un pedido asignado y ruta recalculada. |

## Archivos que la componen

| Archivo (despacho-service) | Qué hace |
|---|---|
| `controller/PlanificacionController.java` | `POST /api/despacho/planificaciones` y `/confirmacion` (reintento). |
| `service/PlanificacionService.java` | Orquesta el proceso: pedidos pendientes, distancias, heurística, persistencia y confirmación (saga). |
| `service/planificacion/PlanificadorRutas.java` | La heurística: decide qué pedido va en qué ruta y en qué orden. Java puro. |
| `service/planificacion/CalculadoraRuta.java` | Simula una ruta y verifica las cinco restricciones. |
| `service/planificacion/MatrizDistancias.java` | Distancias y tiempos entre todos los puntos, con la fuente de cada dato. |
| `service/planificacion/MotivoNoAsignable.java` | Motivos concretos por los que un pedido no entra en ninguna ruta. |
| `service/planificacion/*Plan.java` | Records inmutables con lo mínimo que necesita la heurística (vehículo, repartidor, parada, ruta). |
| `service/DistanciaService.java` | Estrategia de distancias: caché → OSRM → caché vencida → estimación. |
| `client/OsrmClient.java` | Traduce HTTP/JSON de OSRM a Java. No toma decisiones. |
| `model/DistanciaCache.java` | Tabla de caché de distancias ya consultadas. |
| `service/RutaRecalculador.java` | Quita paradas de una ruta y recalcula km, tiempos y costo. |
| `PlanificadorRutasTest (11), DistanciaServiceTest (5), OsrmClientTest (3)` | Tests de la heurística y de la estrategia de distancias. |

## El proceso, paso a paso

| Paso | Qué pasa | Si falla |
|---|---|---|
| 0 | Se borran las rutas en BORRADOR de un intento anterior. | — |
| 1 | Se cargan el centro y los vehículos y repartidores libres ese día. | Centro inexistente → 404; inactivo → 409 |
| 2 | Se piden a pedidos-service los pedidos RECIBIDOS del centro. | pedidos no responde → 503, no se planifica |
| 3 | Se arma la matriz de distancias entre el centro y todos los destinos. | OSRM caído → se usa caché o estimación, con advertencia |

| Paso | Qué pasa | Si falla |
|---|---|---|
| 4 | La heurística arma las rutas y la lista de no asignables. | — |
| 5 | Las rutas se guardan en estado BORRADOR. | Otro despachante a la vez → 409 |
| 6 | Se le pide a pedidos-service que pase los pedidos a ASIGNADO o NO_ASIGNABLE. Las rutas pasan a CONFIRMADA. | pedidos no responde → quedan en BORRADOR y se reintenta la confirmación |

## Fragmentos clave

despacho-service/.../service/planificacion/PlanificadorRutas.java

```
public static final Comparator<PedidoPendienteDTO> ORDEN_DE_ATENCION = Comparator
        .comparing((PedidoPendienteDTO p) -> p.prioridad() == Prioridad.URGENTE ? 0 : 1)
        .thenComparing(PedidoPendienteDTO::ventanaHasta)
        .thenComparing(PedidoPendienteDTO::ventanaDesde)
        .thenComparing(PedidoPendienteDTO::id);
```

RN07: primero los urgentes; dentro de cada prioridad, el pedido cuya ventana cierra antes (el que menos margen tiene). El desempate por id hace que el resultado sea siempre el mismo con los mismos datos.

despacho-service/.../service/planificacion/PlanificadorRutas.java

```
for (PedidoPendienteDTO pedido : ordenados) {
    Optional<MotivoNoAsignable> descarte = prefiltrar(pedido, vehiculos, params);
    if (descarte.isPresent()) { noAsignados.add(...); continue; }

    Opcion mejor = null;
    // a) insertar en una ruta ya abierta, en cada posición posible
    for (RutaEnConstruccion ruta : rutas) {
        for (int pos = 0; pos <= ruta.secuencia.size(); pos++) {
            List<PedidoPendienteDTO> candidata = new ArrayList<>(ruta.secuencia);
            candidata.add(pos, pedido);
            EvaluacionRuta ev = calculadora.evaluar(candidata, ruta.vehiculo, ruta.repartidor, params, matriz);
            if (!ev.factible()) { rechazos.merge(ev.violacion(), 1, Integer::sum); continue; }
            double costo = (ev.kmTotal() - ruta.evaluacion.kmTotal()) * ruta.vehiculo.costoPorKm();
            mejor = elegir(mejor, new Opcion(ruta, null, null, candidata, ev, costo, false));
        }
    }
    // b) abrir una ruta nueva con un par vehículo/repartidor libre
    for (VehiculoPlan v : vehiculosLibres) {
        for (RepartidorPlan r : repartidoresLibres) {
            EvaluacionRuta ev = calculadora.evaluar(List.of(pedido), v, r, params, matriz);
            if (!ev.factible()) { rechazos.merge(ev.violacion(), 1, Integer::sum); continue; }
            double costo = ev.kmTotal() * v.costoPorKm();
            mejor = elegir(mejor, new Opcion(null, v, r, ..., ev, costo, v.tieneFrio() && !pedido.requiereFrio()));
        }
    }
    if (mejor == null) { noAsignados.add(noAsignable(pedido, rechazos)); continue; }
    // ... aplica la mejor opción; si abrió una ruta, el vehículo y el repartidor dejan de estar libres
}
```

Heurística de **inserción más barata**. Para cada pedido se prueban todas las opciones, se descartan las que violan una restricción y se elige la de menor costo (km adicionales × costo por km). A igual costo: no gastar un vehículo refrigerado en un

pedido que no lo necesita, y preferir sumar a una ruta existente. Si no hay opción factible, queda NO_ASIGNABLE con el motivo más frecuente (RN10).

despacho-service/.../service/planificacion/CalculadoraRuta.java

```
// 1 y 2: frío y capacidad (no dependen del orden)
for (PedidoPendienteDTO p : secuencia) {
    if (p.requiereFrio() && !vehiculo.tieneFrio() && violacion == null) {
        violacion = MotivoNoAsignable.SIN_VEHICULO_REFRIGERADO;
    }
    peso += p.pesoKg();
    volumen += p.volumenM3();
}
if (violacion == null && (peso > vehiculo.pesoMaxKg() + 1e-9 || volumen > vehiculo.volMaxM3() + 1e-9)) {
    violacion = MotivoNoAsignable.CAPACIDAD_AGOTADA;
}
// 3: simulación en el tiempo
LocalDateTime reloj = params.fecha().atTime(repartidor.turnoInicio());
for (PedidoPendienteDTO p : secuencia) {
    reloj = reloj.plusSeconds(matriz.segundos(anterior, destino));
    if (reloj.isBefore(p.ventanaDesde())) {
        reloj = p.ventanaDesde();           // llega antes: espera a que abra la ventana
    }
    if (reloj.isAfter(p.ventanaHasta()) && violacion == null) {
        violacion = MotivoNoAsignable.VENTANA_HORARIA_INALCANZABLE;
    }
    etas.add(reloj);
    reloj = reloj.plusMinutes(params.tiempoServicioMin());
    anterior = destino;
}
// 4: turno      -> el regreso al centro debe ser antes del fin del turno  (EXCEDE_TURNO)
// 5: autonomía  -> km totales, incluida la vuelta, <= autonomía          (EXCEDE_AUTONOMIA)
```

RN08: una ruta es válida solo si cumple **todo a la vez**: capacidad de peso y volumen, frío, ventana horaria de cada parada, turno del repartidor y autonomía del vehículo. La simulación avanza un reloj parada por parada y de ahí sale también la hora estimada de llegada (ETA).

despacho-service/.../service/DistanciaService.java

```
// 1) caché vigente (una sola consulta)
if (c != null && c.getFecha().isAfter(limiteVigencia)) { ... fuente[i][j] = FuenteDistancia.CACHE; }

// 2) OSRM para lo que falta
if (hayFaltantes(fuente) && osrm.habilitado()) {
    consultarOsrm(puntos, metros, segundos, fuente, cache, ahora);   // lo obtenido se guarda en la caché
}

// 3) y 4) fallback
DistanciaCache vieja = cache.get(clave(puntos.get(i), puntos.get(j)));
if (vieja != null) {
    fuente[i][j] = FuenteDistancia.CACHE_VENCIDA;
} else {
    double km = puntos.get(i).distanciaHaversineKm(puntos.get(j)) * config.factorRodeo();
    metros[i][j] = (int) Math.round(km * 1000);
    segundos[i][j] = (int) Math.round(km / config.velocidadPromedioKmh() * 3600);
    fuente[i][j] = FuenteDistancia.ESTIMADA;
}
```

RN18: la planificación nunca se cae por culpa de la API externa. Orden: caché vigente (30 días) → OSRM → caché vencida → estimación (línea recta × 1,3 a 30 km/h). La respuesta informa de dónde salió cada distancia y marca la ruta como `estimada` para que el despachante la revise. Si OSRM falla una vez, no se insiste en esa operación.

despacho-service/.../service/PlanificacionService.java

```
try {
    respuesta = pedidosClient.confirmarAsignacion(
            new ConfirmacionAsignacionRequest(centroId, fecha, rutas, noAsignables));
} catch (PedidosNoDisponibleException e) {
    log.warn("pedidos-service no respondió la confirmación; {} rutas quedan en BORRADOR", rutaIds.size());
    return new Confirmacion(EstadoPlanificacion.PENDIENTE_CONFIRMACION, mapear(rutaIds), List.of(),
            List.of("pedidos-service no respondió: las rutas quedaron en BORRADOR. "
                    + "Reintentar con POST /api/despacho/planificaciones/confirmacion"));
}
// ... las paradas de pedidos que pedidos-service rechazó (p. ej. cancelados mientras tanto)
//     se quitan de la ruta y se recalcula; el resto pasa a CONFIRMADA
```

RN13, la **saga**: como cada servicio tiene su base, no hay una transacción que abarque a los dos. Paso 1: despacho guarda las rutas en BORRADOR (el repartidor no las ve). Paso 2: pedidos confirma en una sola transacción. Si el paso 2 no se puede hacer, nada queda inconsistente: se reintenta.

## Conceptos clave

- **Problema de ruteo de vehículos con ventanas horarias (VRPTW):** encontrar el óptimo es NP-difícil. Se usa una **heurística**: no garantiza el mínimo, pero da una solución buena, rápida, explicable y que siempre respeta las restricciones.
- **Factibilidad antes que costo:** primero se descarta lo que no se puede; entre lo que se puede, lo más barato.
- **Java puro:** `PlanificadorRutas` y `CalculadoraRuta` no dependen de Spring ni de la base. Reciben datos y devuelven datos, por eso se prueban con tests unitarios simples.
- **Saga:** secuencia de pasos locales con una forma de recuperarse si uno falla, en lugar de una transacción distribuida.
- **Transacciones cortas:** las llamadas HTTP (pedidos, OSRM) se hacen fuera de la transacción de base (`TransactionTemplate`), para no retener una conexión mientras se espera a otro servicio.

- **Caché y degradación:** la caché ahorra llamadas a OSRM; los *fallback* permiten seguir funcionando con peor calidad de dato, avisando.
- **RN11:** un vehículo y un repartidor hacen una sola ruta por día. Lo aplica la heurística y lo asegura la base con restricciones únicas (`vehiculo_id + fecha`, `repartidor_id + fecha`).
- **RN12:** no se planifican fechas pasadas; dos planificaciones simultáneas del mismo centro y fecha chocan con esas restricciones únicas y la segunda recibe 409.

## Preguntas probables

- **¿La solución es óptima?** No, y no pretende serlo. Es una heurística de inserción más barata. Lo que sí garantiza es que toda ruta generada cumple las cinco restricciones y que el resultado es determinístico.
- **¿Por qué el pedido urgente no queda primero en la ruta?** RN07 define el orden de **atención**: el urgente elige recursos antes que los demás. Su posición dentro de la ruta la deciden el costo y su ventana horaria.
- **¿Qué pasa si OSRM no responde?** Se usa la caché (aunque esté vencida) y, si no hay, una estimación. La planificación sale igual, con una advertencia y las rutas marcadas como estimadas.
- **¿Qué pasa si se cancela un pedido mientras se está planificando?** En la confirmación, pedidos-service lo devuelve como rechazado; despacho quita esa parada, recalcula la ruta y avisa en la respuesta.
- **¿Por qué OSRM y no Google Maps?** Google exige activar facturación con tarjeta aun para la cuota gratuita. OSRM es la alternativa propuesta por la cátedra, con datos de OpenStreetMap y sin API key. Está registrado como desvío en el README.
- **¿Cuál fue la parte más difícil?** Verificar todas las restricciones a la vez (por eso existe `CalculadoraRuta` como simulador) y mantener consistentes dos bases sin transacción distribuida (por eso la saga).

# Parte 4 · Nacho: Ejecución de la ruta y colaboración entre servicios

| Presenta | **Nacho** |
|---|---|
| Objetivo | Explicar qué hace el repartidor en la calle y cómo se avisan los dos servicios entre sí sin perder información cuando uno está caído. |
| Criterios de la rúbrica | INT-01 (12 pts); aporta a NEG-01 y NEG-02. |
| Demo en Postman | Apartado **4. Ejecución de la ruta (repartidor)**: mi ruta, ruta ajena rechazada, inicio, pedidos EN_CAMINO, entrega y un intento fallido. |

## Archivos que la componen

| Archivo | Qué hace |
|---|---|
| `despacho · controller/MiRutaController.java` | Endpoints del repartidor. La identidad sale del token. |
| `despacho · service/EjecucionRutaService.java` | Iniciar la ruta y registrar el resultado de cada parada. |
| `despacho · model/Ruta.java, Parada.java, EstadoRuta.java` | Ruta con paradas ordenadas y sus estados. |
| `despacho · model/NotificacionPedido.java` | **Outbox**: avisos pendientes de enviar a pedidos-service. |
| `despacho · service/NotificacionPedidoService.java` | Envía los avisos del outbox y los reintenta. |
| `despacho · client/PedidosClient.java` | Cliente HTTP de la API interna de pedidos; clasifica los fallos. |
| `despacho · config/RestClientConfig.java` | Clientes HTTP con timeouts y token de servicio. |
| `despacho · controller/internal/InternalDespachoController.java` | API interna que usa pedidos (consultar centro, quitar pedido de la ruta). |
| `despacho · service/CancelacionService.java` | Quita de la ruta un pedido que se va a cancelar. |
| `pedidos · controller/PedidoInternoController.java` | API interna que usa despacho (pendientes, asignaciones, en camino, resultado). |
| `pedidos · client/DespachoClient.java, config/RestClientConfig.java` | Cliente HTTP hacia despacho, con token de servicio. |
| `pedidos · service/PedidoService.java (parte interna)` | `confirmarAsignacion`, `registrarSalida`, `registrarResultado`: idempotentes. |

## Quién llama a quién, y por qué

| Proceso | Inicia | Llamada | Si el otro no responde |
|---|---|---|---|
| Alta de pedido | pedidos | `GET /internal/centros/{id}` | 503: el pedido no se crea |
| Planificación | despacho | `GET /internal/pedidos/pendientes` | 503: no se planifica |
| Confirmación | despacho | `POST /internal/pedidos/asignaciones` | Rutas en BORRADOR; se reintenta |
| Inicio y resultado de ruta | despacho | `POST /internal/pedidos/{id}/en-camino` y `/resultado-entrega` | **Outbox**: se guarda y se reintenta cada 15 s |

| Proceso | Inicia | Llamada | Si el otro no responde |
|---|---|---|---|
| Cancelar un ASIGNADO | pedidos | `DELETE /internal/paradas/pedido/{id}` | 503: no se cancela. Si ya salió: 409 → 422 |

**Regla de fondo:** pedidos-service es dueño del **estado del pedido**; despacho-service es dueño de la **ruta**. Despacho informa *qué ocurrió* («salió», «se entregó», «falló») y pedidos decide la transición.

## Fragmentos clave

despacho-service/.../service/EjecucionRutaService.java

```
@Transactional
public RutaResponse iniciar(String keycloakUserId, Long rutaId) {
    Ruta ruta = rutaDelRepartidor(keycloakUserId, rutaId);
    if (ruta.getEstado() != EstadoRuta.CONFIRMADA) {
        throw new ReglaNegocioException("RUTA_NO_CONFIRMADA", "Sólo se puede iniciar una ruta CONFIRMADA ...");
    }
    if (ruta.getFecha().isAfter(LocalDate.now(clock))) {
        throw new ReglaNegocioException("RUTA_FUTURA", "... todavía no puede iniciarse");
    }
    ruta.setEstado(EstadoRuta.EN_CURSO);
    List<NotificacionPedido> notificaciones = ruta.getParadas().stream()
            .map(p -> NotificacionPedido.enCamino(p.getPedidoId(), ruta.getId(), ahora))
            .toList();
    notificacionRepository.saveAll(notificaciones);                // misma transacción que el cambio de la ruta
    eventos.publishEvent(new NotificacionesRegistradasEvent(notificaciones.size()));
    return DespachoMapper.aResponse(ruta);
}

/** Autorización por propiedad: el repartidor sólo opera SU ruta. */
private Ruta rutaDelRepartidor(String keycloakUserId, Long rutaId) {
    Ruta ruta = rutaRepository.findConRecursos(rutaId)
            .orElseThrow(() -> RecursoNoEncontradoException.de("Ruta", rutaId));
    if (!ruta.getRepartidor().getKeycloakUserId().equals(keycloakUserId)) {
        throw new AccessDeniedException("La ruta " + rutaId + " no está asignada al usuario autenticado");
    }
    return ruta;
}
```

> RN14. Acá **no se llama a pedidos-service**: el cambio de la ruta y el aviso pendiente se guardan juntos. Si pedidos está caído, el repartidor sigue trabajando. El control de propiedad (403 si la ruta es de otro) está en el servicio porque depende de datos.

despacho-service/.../service/NotificacionPedidoService.java

```
@Async
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void alRegistrarse(NotificacionesRegistradasEvent evento) {
    procesarPendientes();                                  // enseguida, en otro hilo
}

@Scheduled(fixedDelayString = "${despacho.notificaciones.intervalo-ms}", ...)
public void reintentoPeriodico() {
    procesarPendientes();                                  // cada 15 s, lo que haya quedado
}

private boolean enviar(NotificacionPedido n) {
    n.setIntentos(n.getIntentos() + 1);
    try {
        switch (n.getTipo()) {
            case EN_CAMINO -> pedidosClient.notificarEnCamino(n.getPedidoId(), ...);
            case RESULTADO_ENTREGA -> pedidosClient.notificarResultado(n.getPedidoId(), ...);
        }
        n.setEstado(EstadoNotificacion.ENVIADA);
        return true;
    } catch (PedidosRechazoException e) {                  // pedidos dijo que no: no reintentar
        n.setEstado(EstadoNotificacion.ERROR);
        return true;
    } catch (PedidosNoDisponibleException e) {             // transitorio: cortar y reintentar después
        if (n.getIntentos() >= props.notificaciones().maxIntentos()) {
            n.setEstado(EstadoNotificacion.ERROR);
        }
        return false;
    }
}
```

RN17, patrón **outbox**. El aviso sale después del commit (solo se avisa lo que quedó guardado), en segundo plano. Los pendientes se procesan en orden de id y, si pedidos no responde, se corta la vuelta: así «en camino» siempre llega antes que el resultado. Lo que pedidos rechaza queda en ERROR para que lo vea el despachante (`GET /api/despacho/notificaciones?estado=ERROR`).

despacho-service/.../client/PedidosClient.java

```
private <T> T invocar(String operacion, Supplier<T> llamada) {
    try {
        return llamada.get();
    } catch (HttpClientErrorException e) {
        if (status.value() == 401 || status.value() == 403) {
            // Problema de credenciales entre servicios: no es culpa del pedido
            throw new PedidosNoDisponibleException("Error de autenticación entre servicios", e);
        }
        throw new PedidosRechazoException(status.value(), "pedidos-service rechazó '" + operacion + "': ...");
    } catch (RuntimeException e) {
        // red, timeout, 5xx, error al obtener el token de Keycloak
        throw new PedidosNoDisponibleException("pedidos-service no disponible", e);
    }
}
```

Todos los fallos se reducen a dos tipos, porque quien llama actúa distinto: **no disponible** (transitorio → reintentar) y **rechazo** (404/409/422 → no reintentar).

despacho-service/.../config/RestClientConfig.java

```
ClientHttpRequestInterceptor bearer = (request, body, execution) -> {
    OAuth2AuthorizeRequest authorizeRequest = OAuth2AuthorizeRequest
            .withClientRegistrationId(REGISTRACION_KEYCLOAK)        // "logistica-service"
            .principal(REGISTRACION_KEYCLOAK)
            .build();
    OAuth2AuthorizedClient client = manager.authorize(authorizeRequest);
    request.getHeaders().setBearerAuth(client.getAccessToken().getTokenValue());
    return execution.execute(request, body);
};
return RestClient.builder()
        .baseUrl(props.pedidos().baseUrl())
        .requestFactory(requestFactory(props.pedidos().connectTimeout(), props.pedidos().readTimeout()))
        .requestInterceptor(bearer)
        .build();
```

Cada llamada entre servicios lleva un token propio del servicio (*client credentials*, rol SERVICE), que Spring pide a Keycloak, guarda y renueva solo. No se reenvía el token del usuario: lo interno no depende de roles humanos. Los timeouts (2 s de conexión, 5 s de lectura) evitan que un servicio colgado cuelgue al otro.

pedidos-service/.../service/PedidoService.java

```
/** El repartidor salio con el pedido: EN_CAMINO. */
@Transactional
public EstadoActualDTO registrarSalida(Long id, String servicio) {
    Pedido pedido = buscar(id);
    if (pedido.getEstado() == EstadoPedido.EN_CAMINO) {
        return EstadoActualDTO.from(pedido);                // ya estaba: no es un error
    }
    pedido.cambiarEstado(EstadoPedido.EN_CAMINO, "Salida del centro de distribucion", servicio);
    return EstadoActualDTO.from(pedido);
}
```

**Idempotencia:** el outbox puede enviar el mismo aviso dos veces (por ejemplo, si pedidos procesó el aviso pero la respuesta se perdió). Recibirlo de nuevo no cambia nada ni da error.

despacho-service/.../service/CancelacionService.java

```
@Transactional
public void quitarPedido(Long pedidoId) {
    // ... busca las rutas EN_CURSO que tienen ese pedido con la parada PENDIENTE
    if (enLaCalle) {
        throw new ReglaNegocioException("PEDIDO_EN_CAMINO",
                "El pedido " + pedidoId + " ya salió a la calle y no puede cancelarse");      // -> 409
    }
    List<Ruta> rutas = rutaRepository.findConPedidoEnEstados(pedidoId,
            EnumSet.of(EstadoRuta.BORRADOR, EstadoRuta.CONFIRMADA));
    for (Ruta ruta : rutas) {
        recalculador.quitarParadas(ruta, List.of(pedidoId), null);       // recalcula km, tiempos y costo
    }
}
```

> RN06. Para cancelar un pedido ASIGNADO, pedidos primero le pide a despacho que lo saque de la ruta. Si ya salió a la calle, despacho responde 409 y pedidos devuelve 422 al cliente. Si el pedido no estaba en ninguna ruta, responde 204 igual.

## Estados de la ruta y de la parada

| Entidad | Estados | Reglas |
|---|---|---|
| Ruta | BORRADOR → CONFIRMADA → EN_CURSO → FINALIZADA | El repartidor solo ve rutas confirmadas; solo inicia una CONFIRMADA que no sea futura; pasa a FINALIZADA cuando todas las paradas tienen resultado (RN15). |
| Parada | PENDIENTE → ENTREGADA o FALLIDA | Solo se registra con la ruta EN_CURSO y la parada PENDIENTE; FALLIDA exige motivo; no se puede registrar dos veces (409). |

## Conceptos clave

- **Comunicación sincrónica vs. asincrónica:** sincrónica cuando se necesita la respuesta para seguir (validar el centro, pedir pendientes); asincrónica con outbox cuando la operación no puede depender del otro servicio (la calle).
- **Outbox:** guardar el aviso en la propia base, en la misma transacción que el cambio, y enviarlo después. Garantiza que no se pierda.
- **Entrega «al menos una vez» + idempotencia:** como un aviso puede repetirse, quien lo recibe tiene que tolerar duplicados.
- **Consistencia eventual:** por unos segundos el pedido puede seguir ASIGNADO aunque la ruta ya esté EN_CURSO. Por eso la colección de Postman espera 3 segundos antes de verificar EN_CAMINO.
- **Contrato:** los DTO de `dto/internal` definen lo que viaja entre servicios. Cada servicio tiene su propia copia; no se comparten entidades ni bases.
- **Timeouts:** toda llamada saliente tiene tiempo máximo de conexión y de lectura.
- **Trazabilidad:** cada cambio de estado guarda quién lo hizo: el `sub` del usuario, `SISTEMA`, o el client id del servicio (`azp` del token).

## Preguntas probables

- **¿Qué pasa si pedidos-service se cae mientras el repartidor está entregando?** El repartidor no se entera: despacho guarda el resultado y el aviso en su base. El envío se reintenta cada 15 segundos y los pedidos se ponen al día cuando pedidos vuelve.
- **¿Por qué no usar una cola de mensajes (RabbitMQ, Kafka)?** Para el alcance del TP, el outbox en tabla da la misma garantía de no perder avisos sin sumar otra pieza de infraestructura. Con más volumen, el paso natural es una cola.

- **¿Cómo evitan procesar dos veces el mismo aviso?** No se evita el reenvío; se hace inofensivo. Los endpoints internos de pedidos son idempotentes: si el pedido ya está en el estado pedido, responden OK sin cambiar nada.
- **¿Cómo se demuestra un error de integración controlado?** Deteniendo pedidos-service y registrando una parada: la respuesta es 200, el aviso queda PENDIENTE y se envía solo al volver a levantar el servicio. O creando un pedido con despacho detenido: 503 con mensaje claro.
- **¿Por qué el repartidor recibe 403 y no 404 con una ruta ajena?** Si la ruta no existe es 404; si existe pero es de otro, 403. El id del repartidor sale del token, así que no puede hacerse pasar por otro cambiando la URL.
- **¿Por qué pedidos decide la transición y no despacho?** Porque el estado del pedido es de pedidos. Si despacho dijera «pasalo a RECIBIDO», tendría que conocer la regla de reintentos, que es del pedido. Despacho solo informa el hecho.

# Parte 5 · Tomi: Seguimiento, mails, persistencia, despliegue y pruebas

| Presenta | **Tomi** |
|---|---|
| Objetivo | Explicar cómo se sigue un pedido y se avisa al cliente, dónde se guardan los datos, cómo se levanta el sistema completo y cómo se comprueba que funciona. |
| Criterios de la rúbrica | INF-01 (10 pts), API-01 (5 pts) y ALC-01 (8 pts); aporta a TEC-01 (pruebas). |
| Demo en Postman | Apartado **5. Seguimiento, mails y seguridad**: estado final, historial (un mail por cambio), distancia con OSRM, avisos con error y accesos rechazados. |

## Archivos que la componen

| Archivo | Qué hace |
|---|---|
| `pedidos · model/CambioEstadoListener.java` | Detecta cada cambio de estado guardado y publica un evento. |
| `pedidos · service/NotificadorMail.java` | Escucha el evento y manda el mail en HTML al cliente. |
| `pedidos · controller/PedidoController.java (/{id}/cambios)` | Historial del pedido. |
| `despacho · controller/NotificacionController.java, DistanciaController.java` | Supervisión de avisos entre servicios y consulta de distancias. |
| `*/src/main/resources/application.yml` | Configuración de cada servicio; todo valor se puede pisar por variable de entorno. |
| `infra/init-db.sql, reset-datos.sql` | Crea las tres bases con sus usuarios; vuelve los datos al estado inicial. |
| `despacho · src/main/resources/data.sql` | Datos semilla: centros, vehículos y repartidores. |
| `*/Dockerfile` | Imagen de cada servicio (compilación + ejecución). |
| `docker-compose.yml, docker-compose.override.yml, .env.example` | Despliegue completo y su configuración. |
| `postman/TPI-Logistica.postman_collection.json, Local.postman_environment.json` | Colección ejecutable: 5 apartados, 39 requests, 73 verificaciones. |
| `*/src/test/java/**` | Tests unitarios, de integración y de seguridad (más de 90). |
| `README.md` | Arquitectura, puesta en marcha, trazabilidad y desvíos. |

## Fragmentos clave

pedidos-service/.../model/CambioEstadoListener.java

```
public class CambioEstadoListener {

    /** Datos del cambio, sin entidades: el evento se consume en otro hilo, fuera de esta sesión. */
    public record CambioRegistrado(Long pedidoId, EstadoPedido anterior, EstadoPedido nuevo, String motivo) {
    }

    @PostPersist
    void alRegistrar(CambioEstado cambio) {
        eventos.publishEvent(new CambioRegistrado(cambio.getPedido().getId(),
                cambio.getEstadoAnterior(), cambio.getEstadoNuevo(), cambio.getMotivo()));
    }
}
```

Todo cambio de estado inserta una fila en `cambio_estado`; este listener de JPA es el único lugar que se entera de todos, vengan de donde vengan. Solo publica un evento: no sabe nada de mails.

pedidos-service/.../service/NotificadorMail.java

```
@Async
@TransactionalEventListener
@Transactional(propagation = Propagation.REQUIRES_NEW, readOnly = true)
public void alCambiarEstado(CambioRegistrado cambio) {
    if (!habilitado) { return; }                                  // sin contraseña de Gmail no se envía nada
    Cliente cliente = pedidoRepository.findById(cambio.pedidoId()).orElseThrow().getCliente();
    try {
        MimeMessage mensaje = mailSender.createMimeMessage();
        MimeMessageHelper mail = new MimeMessageHelper(mensaje, true, "UTF-8");
        mail.setFrom(remitente);
        mail.setTo(cliente.getEmail());
        if (copia.length > 0) {
            mail.setBcc(copia);                                   // copia oculta a las casillas de prueba
        }
        mail.setSubject("Pedido #" + cambio.pedidoId() + ": " + titulo(cambio.nuevo()));
        mail.setText(cuerpo(cliente.getRazonSocial(), cambio), html(cliente.getRazonSocial(), cambio));
        mailSender.send(mensaje);
    } catch (MailException | MessagingException e) {
        log.warn("No se pudo enviar el mail del pedido {}: {}", cambio.pedidoId(), e.getMessage());
    }
}
```

Tres decisiones: sale **después del commit** (no se avisa algo que se revirtió); es **asíncrono** (el request no espera a Gmail); y **nunca lanza excepciones** (si Gmail falla, el pedido no se ve afectado y queda el aviso en el log). El mail lleva versión HTML y texto plano; lo que viene del usuario se escapa antes de insertarlo en el HTML.

pedidos-service/src/main/resources/application.yml

```
spring:
  datasource:
    url: jdbc:postgresql://${PEDIDOS_DB_HOST:localhost}:${PEDIDOS_DB_PORT:5432}/${PEDIDOS_DB_NAME:pedidos_db}
    username: ${PEDIDOS_DB_USER:pedidos_user}
    password: ${PEDIDOS_DB_PASSWORD:pedidos_pass}
  jpa:
    hibernate:
      ddl-auto: update
  mail:
    host: smtp.gmail.com
    port: 587
    username: ${GMAIL_USER:...}
    password: ${GMAIL_APP_PASSWORD:...}
```

**Configuración por entorno:** `${VARIABLE:valor por defecto}`. En local se usan los valores por defecto; en Docker el compose pisa solo lo que cambia (por ejemplo, `PEDIDOS_DB_HOST=postgres`). El código no se toca.

pedidos-service/Dockerfile

```
# ---- Etapa 1: compilar con Maven ----
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /build
COPY logistica-commons logistica-commons
RUN --mount=type=cache,target=/root/.m2,sharing=locked \
    mvn -q -f logistica-commons/pom.xml -Dmaven.test.skip=true install
COPY pedidos-service pedidos-service
RUN --mount=type=cache,target=/root/.m2,sharing=locked \
    mvn -q -f pedidos-service/pom.xml -Dmaven.test.skip=true package

# ---- Etapa 2: imagen final, solo con el JRE y el JAR ----
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
RUN addgroup -S app && adduser -S app -G app
COPY --from=build /build/pedidos-service/target/*.jar app.jar
USER app
EXPOSE 8081
ENTRYPOINT ["java", "-jar", "app.jar"]
```

**Build en dos etapas:** la primera tiene Maven y el JDK y compila (primero el JAR común); la segunda solo lleva lo necesario para ejecutar. Quien corrige no necesita Java ni Maven instalados. El contenedor corre con un usuario sin privilegios.

docker-compose.yml

```
services:
  postgres:
    image: postgres:17-alpine
    volumes:
      - logistica-postgres-data:/var/lib/postgresql/data
      - ./infra/init-db.sql:/docker-entrypoint-initdb.d/init-db.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -h 127.0.0.1 -U postgres"]

  keycloak:
    image: quay.io/keycloak/keycloak:26.4
    command: start-dev --import-realm
    environment:
      KC_HTTP_PORT: 8180
      KC_HOSTNAME: http://localhost:8180
      KC_HOSTNAME_BACKCHANNEL_DYNAMIC: "true"
    ports:
      - "8180:8180"

  pedidos-service:
    build:
      context: .
      dockerfile: pedidos-service/Dockerfile
    environment:
      PEDIDOS_DB_HOST: postgres
      DESPACHO_SERVICE_URL: http://despacho-service:8082
      KEYCLOAK_ISSUER_URI: http://localhost:8180/realms/logistica
      KEYCLOAK_JWK_SET_URI: http://keycloak:8180/realms/logistica/protocol/openid-connect/certs
    depends_on:
      postgres:
        condition: service_healthy

  api-gateway:
    ports:
      - "8080:8080"
    depends_on:
      pedidos-service:
        condition: service_healthy
      despacho-service:
        condition: service_healthy

networks:
  logistica-net:
    driver: bridge
```

Fragmento resumido. Dentro de la red los contenedores se llaman por nombre de servicio (`postgres`, `keycloak`, `despacho-service`), no por `localhost`. Solo se publican el gateway (8080) y Keycloak (8180); en desarrollo, el archivo `override` publica también los servicios y la base. El emisor del token es siempre la URL pública de Keycloak; las claves se bajan por la red interna.

postman/TPI-Logistica.postman_collection.json (script de un request)

```
pm.test('HTTP 201', () => pm.expect(pm.response.code).to.be.oneOf([201]));
const p = pm.response.json();
pm.test('Estado RECIBIDO', () => pm.expect(p.estado).to.eql('RECIBIDO'));
pm.environment.set('pedidoUrgenteId', p.id);
```

> Cada request **verifica** su resultado y guarda en el entorno los ids y tokens que usan los siguientes. Por eso la colección se corre completa sin copiar nada a mano.

## Persistencia

| Base | Dueño | Tablas principales |
|---|---|---|
| `pedidos_db` | pedidos-service | `cliente`, `pedido`, `cambio_estado` |
| `despacho_db` | despacho-service | `centro_distribucion`, `vehiculo`, `repartidor`, `ruta`, `parada`, `notificacion_pedido`, `distancia_cache` |
| `keycloak_db` | Keycloak | Usuarios, roles y clients |

- Una instancia de PostgreSQL con **una base y un usuario por servicio**: ninguno puede leer la base del otro.
- Las tablas las crea Hibernate al arrancar (`ddl-auto: update`). Alcanza para el TP; en un entorno real se usarían migraciones versionadas (Flyway o Liquibase).
- La semilla (`data.sql`) es idempotente: usa `ON CONFLICT DO NOTHING`, así que se puede ejecutar en cada arranque.
- Reglas aseguradas por la base: restricciones únicas de `ruta` (un vehículo y un repartidor por día), CUIT y usuario únicos en `cliente`.
- `reset-datos.sql` borra pedidos, clientes y rutas, y conserva la semilla y la caché de distancias. Hace falta para repetir la demo el mismo día (RN11).

## Pruebas

| Tipo | Clases | Qué comprueban |
|---|---|---|
| Reglas del modelo | `PedidoTest`, `CoordenadaTest` | Transiciones, cancelación, reintentos, distancia Haversine. |
| Casos de uso | `PedidoServiceTest`, `ClienteServiceTest` | Alta con sus rechazos, permisos por dato, idempotencia de lo interno. |
| Heurística | `PlanificadorRutasTest` | Prioridad, capacidad, frío, ventanas, turno, cobertura, costo. |
| Integración externa | `DistanciaServiceTest`, `OsrmClientTest` | Caché, OSRM, caché vencida y estimación. |
| Seguridad | `PedidoControllerSecurityTest`, `DespachoSecurityTest` | Sin token, rol incorrecto, recurso ajeno, `/internal`. |
| Punta a punta en memoria | `NotificadorMailTest` | Transacción real (H2) → evento → mail; ninguno si hay rollback. |
| Funcionales | Colección de Postman | El proceso completo contra el sistema levantado, con casos negativos. |

## Conceptos clave

- **Evento de dominio:** «el pedido cambió de estado». Quien lo produce no conoce a quien lo consume; mañana se puede sumar otro aviso sin tocar el pedido.
- **AFTER_COMMIT + @Async:** efectos secundarios (mail) después de que el dato quedó firme y sin frenar la respuesta.
- **Imagen vs. contenedor:** la imagen es la plantilla (lo que arma el Dockerfile); el contenedor es una ejecución de esa imagen.
- **Docker Compose:** describe todos los contenedores, su red, sus volúmenes y su orden de arranque en un archivo; se levanta todo con un comando.
- **Volumen:** los datos de PostgreSQL sobreviven a `docker compose down`; se borran con `down -v`.
- **Healthcheck y depends_on:** el gateway arranca cuando los servicios responden `/actuator/health/readiness`; los servicios, cuando PostgreSQL acepta conexiones.
- **Trazabilidad:** el README tiene la matriz compromiso → implementación → evidencia (funcionalidades F01–F12, reglas RN01–RN20) y la lista de desvíos acordados.

## Preguntas probables

- **¿Qué pasa si Gmail no responde?** Nada para el pedido: el envío está fuera de la transacción y captura el error, que queda en el log. No hay reintentos de mail; es un límite conocido y aceptado para el TP.
- **¿Cómo levanto todo desde cero?** `docker compose up --build -d` desde la raíz. Crea las bases, importa el realm de Keycloak, compila los tres servicios y los arranca en orden. Después se corre la colección de Postman.
- **¿Por qué una sola instancia de PostgreSQL y no una por servicio?** Lo que importa para el aislamiento es que cada servicio tenga su base y su usuario, y eso se cumple. Una instancia simplifica el despliegue del TP; separarlas es cambiar una variable de entorno.
- **¿Por qué `ddl-auto: update`?** Simplicidad para el TP. La limitación: no versiona cambios de esquema ni borra columnas. En producción: Flyway o Liquibase con `validate`.
- **¿Las credenciales están en el repositorio?** Los valores por defecto son cuentas de prueba (condición de la cátedra para que corra sin configuración). Todos se pueden reemplazar por variables de entorno o por el archivo `.env`, que no se versiona.
- **¿La colección solo lista endpoints?** No: cada request tiene verificaciones (`pm.test`), encadena ids y tokens, y hay casos negativos (422, 403, 401). Está parametrizada con el entorno `Local`.
- **¿Qué quedó fuera del alcance?** Lo indica la sección «Desvíos» del README: OSRM en lugar de Google Maps, el segundo fallo no se puede mostrar el mismo día (RN11) y se cubre con un test, y OSRM propio queda fuera del compose porque se usa el público.

# Antes de la defensa

## Lista de verificación

- **Probar el despliegue con Docker en una máquina con Docker instalado.** Los Dockerfile y el compose están escritos pero todavía no se ejecutaron: hay que correr `docker compose up --build -d`, esperar a que `api-gateway` quede `healthy` y pasar la colección completa. Es evidencia directa de INF-01.
- Correr la colección completa con la base limpia (`reset-datos.sql`) y confirmar 39 requests sin fallas.
- Confirmar que llegan los mails a las casillas de prueba (revisar también la carpeta de correo no deseado).
- Correr `mvn test` en `logistica-commons`, `pedidos-service` y `despacho-service`.
- Actualizar la línea «Probado» del README y las dos guías en PDF, que todavía describen la colección anterior.
- Ensayar el error de integración controlado: detener pedidos-service, registrar una parada, volver a levantarlo y mostrar que el pedido se actualiza solo.
- Tener abiertos antes de empezar: Postman con el entorno **Local**, Swagger de los dos servicios, el README y el IDE en los archivos de cada parte.

## Límites conocidos

Conviene decirlos antes de que los pregunten.

| Límite | Por qué se aceptó | Cómo se mejoraría |
|---|---|---|
| La heurística no garantiza el óptimo | El problema es NP-difícil; se priorizó una solución correcta y explicable | Mejora local posterior (intercambios entre rutas) |
| Los mails no se reintentan | Un mail perdido no afecta al pedido | Outbox de mails, como el de avisos entre servicios |
| Formato de error distinto en cada servicio | Cada servicio es consistente por dentro | Unificar en `ProblemDetail` desde el JAR común |
| `ddl-auto: update` | Simplicidad para el TP | Migraciones versionadas |
| El segundo intento fallido no se ve en la misma demo | RN11: no queda repartidor libre ese día | Está cubierto por `PedidoTest`; en vivo, al día siguiente |
| OSRM público, sin garantía de disponibilidad | No requiere preparar 3 GB de mapas | OSRM propio en contenedor; solo cambia `OSRM_URL` |

## Cómo responder una pregunta que no es de tu parte

Ubicarla en el recorrido del pedido (tabla «El recorrido de un pedido»), decir qué servicio y qué clase la resuelve, y dar la regla en una frase. Si hace falta el detalle, lo completa quien presenta esa parte. Lo que la rúbrica penaliza es que el dominio técnico sea desigual, no que alguien pida apoyo en un detalle.